

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CƠ BẢN I

THỰC TẬP CƠ SỞ



BÁO CÁO TUẦN

Đề tài:

Xây dựng website tuyển dụng việc làm

Giảng viên hướng dẫn	: Kim Ngọc Bách
Họ và tên sinh viên	: Đỗ Bình Minh
Ngày sinh	: 24/12/2005
Số điện thoại	: 0329279863
Mã sinh viên	: B23DCCN542
Lớp	: D23CQCN10-B
Nhóm lớp	: 11

Hà Nội – 2026

Mục lục

1. Tóm tắt các công việc đã thực hiện trong tuần	3
2. Chi tiết các công việc đã thực hiện trong tuần	3
2.1. Thực hiện các chức năng tìm kiếm, lọc	3
2.1.1. Xử lý bên phía Frontend.....	3
2.1.2. Xử lý bên phía Backend	4
2.1.3. Kết quả	10
3. Kết quả đạt được	10
4. Khó khăn gặp phải.....	11
5. Hướng xử lý.....	11
6. Kế hoạch tuần tiếp theo	11

1. Tóm tắt các công việc đã thực hiện trong tuần

Trong tuần này, em tiếp tục hoàn thiện trang tìm kiếm việc làm của website tuyển dụng. Cụ thể, em đã xây dựng chức năng lấy các điều kiện tìm kiếm từ URL như ngôn ngữ lập trình, thành phố, công ty, từ khóa, cấp bậc và hình thức làm việc.

Ở phía frontend, em sử dụng `useSearchParams` để lấy các tham số tìm kiếm và gọi API `/search` để lấy danh sách công việc phù hợp. Ở phía backend, em xây dựng route và controller xử lý tìm kiếm, kết hợp nhiều điều kiện lọc khác nhau để truy vấn dữ liệu từ MongoDB và trả kết quả về cho giao diện.

2. Chi tiết các công việc đã thực hiện trong tuần

2.1. Thực hiện các chức năng tìm kiếm, lọc

2.1.1. Xử lý bên phía Frontend

```
const company = searchParams.get("company") || "";
const keyword = searchParams.get("keyword") || "";
const position = searchParams.get("position") || "";
const workingForm = searchParams.get("workingForm") || "";
const [jobList, setJobList] = useState<any[]>([]);

useEffect(() => {
  fetch(`${process.env.NEXT_PUBLIC_API_URL}/search?
    language=${language}&city=${city}&company=${company}&keyword=${keyword}&position=${position}&workingForm=${
    workingForm}` , {
    method: "GET"
  })
    .then(res => res.json())
    .then((res) => {
      setJobList(data.jobs);
    })
  }, [language, city, company, keyword, position, workingForm]);
```

Đoạn code này là phần **frontend** lấy các điều kiện lọc trên URL rồi gọi API `search`.

Ý nghĩa từng phần:

- `searchParams.get("company")`: lấy tên công ty trên URL.
- `searchParams.get("keyword")`: lấy từ khóa tìm kiếm.
- `searchParams.get("position")`: lấy vị trí công việc.
- `searchParams.get("workingForm")`: lấy hình thức làm việc.

Ví dụ URL:

/search?keyword=react&city=Hà

Nội&company=FPT&position=Intern&workingForm=Fulltime

Sau đó fetch gọi API backend:

GET

/search?language=...&city=...&company=...&keyword=...&position=...&workingForm=.

..

Backend dựa vào các query này để lọc danh sách job.

Phần này:

}, [language, city, company, keyword, position, workingForm]);

có nghĩa là: **mỗi khi một trong các điều kiện lọc thay đổi thì gọi lại API để lấy danh sách job mới.**

2.1.2. Xử lý bên phía Backend

```
import searchRoutes from "../search.route";
```

Dòng code này dùng để **import router xử lý chức năng tìm kiếm** từ file search.route.

```
import { Router } from "express";
import * as searchController from "../controllers/search.controller";

const router = Router();

router.get("/", searchController.search)

export default router;
```

Đoạn code này là **file route của chức năng tìm kiếm việc làm** trong backend Express.

Ý nghĩa từng phần:

- Router: dùng để tạo nhóm route riêng cho chức năng search.
- searchController: import các hàm xử lý từ file search.controller.
- const router = Router();: tạo router mới.
- router.get("/", searchController.search): khi frontend gọi API dạng GET thì chạy hàm search.

Ví dụ nếu trong file route chính có:

```
app.use("/search", searchRoutes);
```

thì dòng này sẽ tạo API:

GET /search

Khi frontend gọi:

/search?language=React

backend sẽ chuyển sang hàm:

searchController.search

để xử lý tìm kiếm

2.1.2.1. Hiển thị data theo trường language

```
export const search = async (req: Request, res: Response) => {
  const dataFinal = [];

  if(Object.keys(req.query).length > 0) {
    const find: any = {};

    if(req.query.language) {
      find.technologies = req.query.language;
    }

    const jobs = await Job
      .find(find)
      .sort({
        createdAt: "desc"
      })

    for (const item of jobs) {
      const itemFinal = {
        id: item.id,
        companyLogo: "",
        title: item.title,
        companyName: "",
        salaryMin: item.salaryMin,
        salaryMax: item.salaryMax,
        position: item.position,
        workingForm: item.workingForm,
        cityName: "",
        technologies: item.technologies
      };

      const companyInfo = await AccountCompany.findOne({
        _id: item.companyId
      })

      if(companyInfo) {
        itemFinal.companyLogo = `${companyInfo.logo}`;
        itemFinal.companyName = `${companyInfo.companyName}`;

        const city = await City.findOne({
          _id: companyInfo.city
        })

        itemFinal.cityName = `${city?.name}`;

        dataFinal.push(itemFinal);
      }
    }
  }

  res.json({
    code: "success",
    message: "Thành công!",
    jobs: dataFinal
  });
}
```

Đây là **hàm controller xử lý API tìm kiếm job** ở backend.

```
export const search = async (req: Request, res: Response) => {
```

Hàm này chạy khi frontend gọi API:

```
GET /search?language=React
```

Luồng xử lý chính:

1. Tạo mảng kết quả rỗng

```
const dataFinal = [];
```

Mảng này sẽ chứa danh sách job sau khi xử lý xong.

2. Kiểm tra có query trên URL không

```
if(Object.keys(req.query).length > 0) {
```

Ví dụ URL có:

```
/search?language=React
```

thì req.query.language sẽ là "React".

3. Tạo điều kiện tìm kiếm

```
const find: any = {};
```

```
if(req.query.language) {  
  find.technologies = req.query.language;  
}
```

Nếu có language, backend sẽ tìm job có công nghệ tương ứng.

Ví dụ:

```
find = {  
  technologies: "React"  
}
```

4. Tìm job trong MongoDB

```
const jobs = await Job  
  .find(find)  
  .sort({  
    createdAt: "desc"  
  })
```

Nghĩa là: tìm các job phù hợp và sắp xếp job mới nhất lên trước.

5. Duyệt từng job để format dữ liệu trả về

```
for (const item of jobs) {
```

Mỗi job được chuyển thành object mới:

```
const itemFinal = {  
  id: item.id,  
  companyLogo: "",  
  title: item.title,  
  companyName: "",  
  salaryMin: item.salaryMin,  
  salaryMax: item.salaryMax,  
  position: item.position,  
  workingForm: item.workingForm,  
  cityName: "",  
  technologies: item.technologies  
};
```

Mục đích là chỉ lấy những trường cần gửi về frontend.

6. Lấy thêm thông tin công ty

```
const companyInfo = await AccountCompany.findOne({  
  _id: item.companyId  
});
```

Vì trong bảng job chỉ lưu companyId, nên phải tìm sang bảng công ty để lấy:

```
companyLogo  
companyName
```

7. Lấy thêm tên tỉnh/thành phố

```
const city = await City.findOne({  
  _id: companyInfo.city  
});
```

Sau đó gán:

```
itemFinal.cityName = `${city?.name}`;
```

8. Đẩy job đã xử lý vào mảng kết quả

```
dataFinal.push(itemFinal);
```

9. Trả kết quả về frontend

```
res.json({  
  code: "success",  
  message: "Thành công!",  
  jobs: dataFinal  
});
```

Frontend sẽ nhận được jobs, rồi dùng:

```
setJobList(data.jobs);
```

để hiển thị danh sách job.

2.1.2.2. Hiển thị data theo trường city

```
if(req.query.city) {  
  const city = await City.findOne({  
    name: req.query.city  
  })  
  if(city) {  
    const listCompanyInCity = await AccountCompany.find({  
      city: city.id  
    })  
    const listIdCompanyInCity = listCompanyInCity.map(item => item.id);  
    find.companyId = { $in: listIdCompanyInCity };  
  }  
}
```

Đoạn code này dùng để **lọc job theo tỉnh/thành phố**.

```
if(req.query.city) {  
  const city = await City.findOne({  
    name: req.query.city  
  })  
}
```

```
if(city) {  
  const listCompanyInCity = await AccountCompany.find({  
    city: city.id  
  })  
}
```

```
const listIdCompanyInCity = listCompanyInCity.map(item => item.id);
```

```
find.companyId = { $in: listIdCompanyInCity };  
}  
}
```


Giải thích ngắn gọn:

- `if(req.query.city)`: kiểm tra URL có truyền city không.

Ví dụ:

`/search?city=Hà Nội`

- `City.findOne({ name: req.query.city })`: tìm thành phố trong MongoDB theo tên.
- `if(city)`: nếu tìm thấy thành phố thì tiếp tục.
- `AccountCompany.find({ city: city.id })`: tìm các công ty thuộc thành phố đó.
- `.map(item => item.id)`: lấy danh sách ID của các công ty.
- `find.companyId = { $in: listIdCompanyInCity }`: thêm điều kiện tìm job có `companyId` nằm trong danh sách công ty ở thành phố đó.

2.1.2.3. Hiển thị data theo trường company

```
if(req.query.company) {
  const company = await AccountCompany.findOne({
    companyName: req.query.company
  })
  find.companyId = company?.id;
}
```

Đoạn code này dùng để **lọc job theo tên công ty** trong API search.

2.1.2.4. Hiển thị data theo trường keyword

```
if(req.query.keyword) {
  const keywordRegex = new RegExp(`${req.query.keyword}`, "i");
  find.title = keywordRegex;
}
```

Đoạn code này dùng để **lọc job theo từ khóa trong tiêu đề**.

2.1.2.5. Bộ lọc theo cấp bậc

```
if(req.query.position) {
  find.position = req.query.position;
}
```

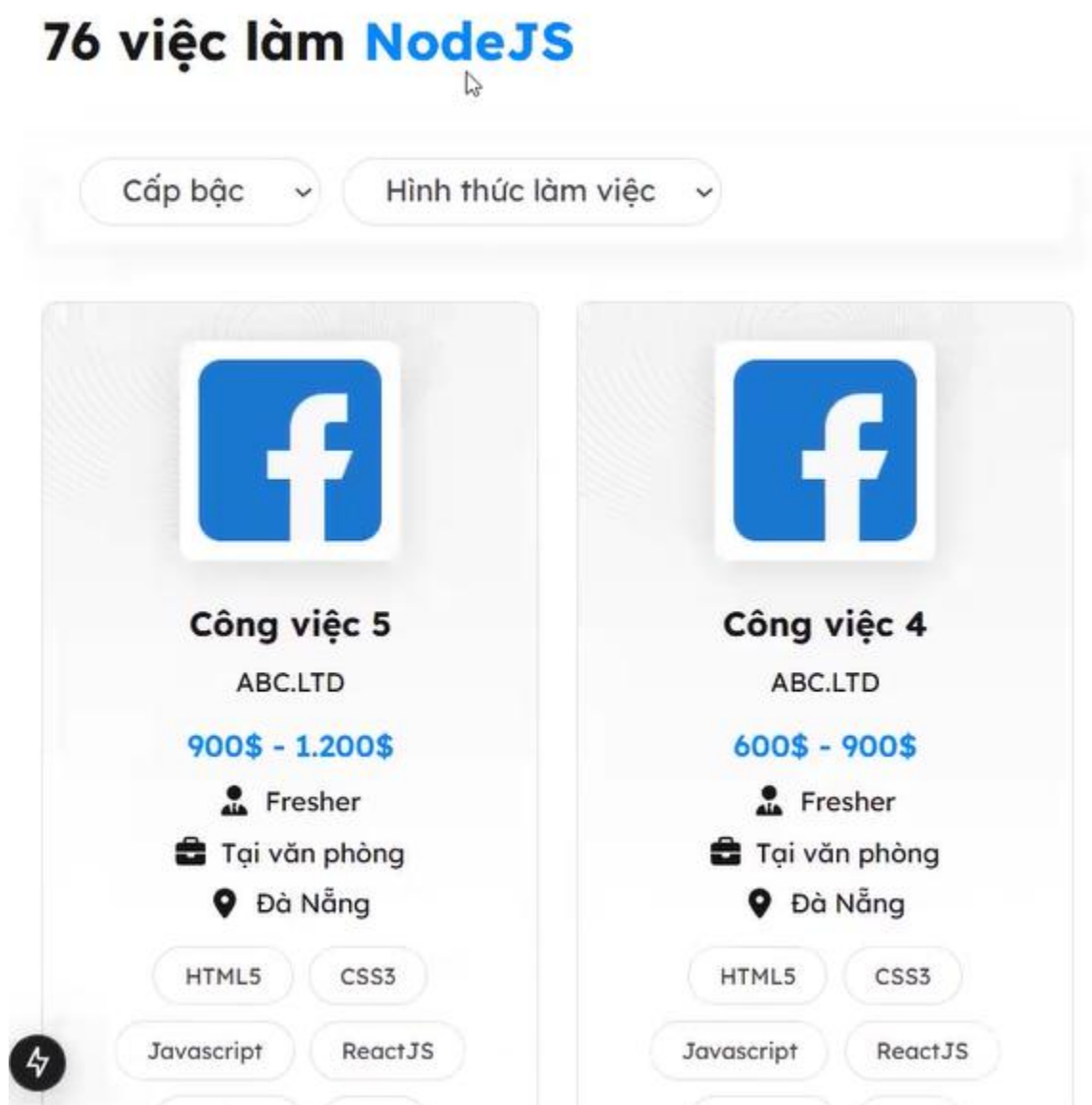
Đoạn code này dùng để **lọc job theo vị trí/chức vụ công việc**.

2.1.2.6. Bộ lọc theo hình thức làm việc

```
if(req.query.workingForm) {
  find.workingForm = req.query.workingForm;
}
```

Đoạn code này dùng để **lọc job theo hình thức làm việc**.

2.1.3. Kết quả



Các công việc đã được lọc và tìm kiếm sẽ hiển thị ra giao diện người dùng.

3. Kết quả đạt được

Trong tuần này, em đã hoàn thiện được chức năng tìm kiếm và lọc công việc trên trang kết quả tìm kiếm.

Người dùng có thể tìm kiếm công việc theo nhiều tiêu chí như ngôn ngữ lập trình, thành phố, tên công ty, từ khóa trong tiêu đề, vị trí công việc và hình thức làm việc. Khi các

tham số trên URL thay đổi, frontend sẽ tự động gọi lại API để cập nhật danh sách công việc mới.

Backend đã xử lý được các điều kiện lọc tương ứng, đồng thời bổ sung thêm thông tin công ty, logo công ty và tên thành phố để dữ liệu trả về đầy đủ hơn cho frontend hiển thị.

4. Khó khăn gặp phải

Trong quá trình thực hiện, em gặp khó khăn khi xử lý nhiều điều kiện lọc cùng lúc trên cùng một API tìm kiếm.

Một số trường dữ liệu không nằm trực tiếp trong bảng công việc, ví dụ thành phố và tên công ty, nên backend phải truy vấn thêm sang bảng City và AccountCompany để lấy đúng dữ liệu liên quan.

Ngoài ra, việc đồng bộ tham số trên URL với dữ liệu hiển thị ở frontend cũng cần xử lý cẩn thận để khi người dùng thay đổi bộ lọc thì danh sách công việc được cập nhật chính xác.

5. Hướng xử lý

Để xử lý các khó khăn trên, em tách từng điều kiện lọc thành từng phần riêng trong controller backend.

Với lọc theo ngôn ngữ lập trình, cấp bậc và hình thức làm việc, em thêm trực tiếp điều kiện vào object find. Với lọc theo thành phố, em tìm thành phố trước, sau đó lấy danh sách công ty thuộc thành phố đó rồi lọc công việc theo companyId. Với lọc theo công ty, em tìm công ty theo tên rồi lấy id để lọc công việc.

Ở frontend, em lấy các tham số từ URL bằng useSearchParams, đưa các giá trị đó vào API /search, sau đó lưu kết quả trả về vào state jobList để render danh sách công việc.

6. Kế hoạch tuần tiếp theo

Trong tuần tiếp theo, em dự kiến tiếp tục hoàn thiện trang tìm kiếm việc làm để giao diện hiển thị đầy đủ và dễ sử dụng hơn.

Cụ thể, em sẽ kiểm tra lại toàn bộ các bộ lọc, xử lý trường hợp không có kết quả tìm kiếm và cải thiện giao diện hiển thị danh sách công việc. Bên cạnh đó, em sẽ tiếp tục phát triển tính năng phân trang cho trang kết quả tìm kiếm đồng thời xây dựng thêm các trang trang form ứng tuyển, danh sách công ty...